

CS585 Summer 2025 Rubric

Q1

[1 point] A primary key (PK) in a table row is like a 'pointer', in the sense that it 'points' to all the non-prime attributes. We don't call that 'pointing' - what is the proper DB terminology for this?

- **1 point: Determination** (or 'functional dependency'). The key is used to determine other (non-prime, ie non-key) values of the row
- **0.5 points:** Some points can be considered for mentioning full functional dependency and arguing that the PK itself comprises the set of determinants.
- **0 points:** Irrelevant answers

[1 point] In theory, what PK-related 'issue' can occur in a single table, and what is the issue called?

- **1 point:** The issue is called '**entity integrity violation**' - happens when entire rows become identical on account of null or replicate (ie identical) PKs.
- **0.5 points:** The student names the issue but does not describe the issue itself. Or the student describes the relevant issue, but does not provide the name of the issue.
- **0 points:** Irrelevant answers

[1 point] In theory, what PK-related 'issue' can occur in two related tables? What is the issue called?

- **1 point:** Foreign keys may refer to an absent or deleted row in the other table, which leads to the **violation of Referential Integrity**.
 - Any other relevant answer that makes sense and is correctly named would yield full credit for the student
- **0.5 points:** The student names the issue but does not describe the issue itself. Or the student describes the relevant issue, but does not provide the name of the issue.
- **0 points:** Irrelevant answers

Q2

What is the issue with 'too much' table normalization? What is the remedy?

(2 pt): The student mentions any content related to "increase query latency/cost/decrease efficiency" - due to excessive JOINS that need to occur in a normalized set of tables

(1pt): The student mentions the remedy is to do table denormalization.

(0pt): Irrelevant answers

Q3

The table format (ie relational model) for storing data is nearly perfect, with only a few drawbacks.

What is a (significant) drawback that JSON can trivially fix? Illustrate with a simple example (showing the problem occurring in a table, and the JSON fix).

Hint - we discussed this in class.

(1 point) Major issue:

Inability to handle hierarchical or nested data structures without resorting to complex join operations or creating numerous related tables; another equally valid answer: **inability to represent multi-valued attributes directly** (ie. in a cell/field, ie row column intersection)

(1 point) example of a relational database where storing hierarchical or nested data become complex, or, an example that shows multi-valued attrs (eg. in the 'degree' column in a student table there is no way to specify 'BS, MS, PhD')

(1 point) Representing the same database using json clearly illustrating how it fixes the issues. Eg in the multivalued attr example, in JSON we would have "degree": ['BS','MS','PhD'], ie. use an array for the multiple values of the attribute.

Q4

Prior to relational tables, digital file systems were used to store and process data, where the significant drawback was 'structural dependency'.

[2 points]

What is 'structural dependency'? Illustrate with a simple/brief example.

[1 point]

The relational (table) format appears to make the problem (of structural dependency) go away - but it never really does! Where does it go, then? Be brief, be specific. Hint: we did cover this!

(1 point) Definition: Program logic is tied to the physical (or positional) layout of a data file. Any change in that layout forces program changes.

Other phrasing: Fields are located by absolute position; if you add, move or resize a field you must rewrite every program that reads the file.

(1 point) Any illustrative examples are fine. E.g. A payroll file stores *EmpID* in bytes 1-5 and *Salary* in bytes 6-11. If a new *MiddleName* field is inserted after *EmpID*, every COBOL routine that assumes *Salary* starts at byte 6 breaks and must be recoded

(1 point) Structural dependency is pushed up to the **logical schema / SQL interface**: changing a table's column order, name, datatype, or splitting/merging tables still breaks any stored procedures, queries, OR-mappers, or application code that reference the old structure. Views

can hide some changes, but without them the dependency now lives in the *application layer* instead of the file layout. Another way to state this: the structural dependency is now at the data representation level, which is not accessible/visible to users [in other words the DB driver for the specific DB handles structurally-dependent data access, while users 'simply' write SQL for this, which offers them positional independence].

(Award 0 pt if the answer simply says "the problem is solved" or gives an unrelated explanation.)

Q5

a. [1 point] Complete these two, name (expand) what they stand for:

2P[]

2P[]

b. [1 point] What do the two '2P?' have in common, ie how are they related?

c. [1 point] What do the two '2P?' have by way of difference, ie how are they not the same (ie. what is each for)?

Q5a

+0.5: 2PL (Two-phase Locking)

+0.5: 2PC (Two-phase Commit)

Q5b

+1: Both are used to maintain correctness when multiple resources or nodes are involved, that need to carry out multiple concurrent transactions.

Q5c

+1: 2PL is a concurrency control mechanism, 2PC is a distributed commit protocol

(or: 2PL is for serializability vs 2PC is for atomic commit)

(or: 2PL is for a single node, whereas 2PC is for an entire collection of nodes in a distributed DB)

Q6 [verify De Morgan's Law]

Set A = {C, D, E, F}, Set B = {E, F, G, H}

Prove/verify that $\neg(A \cup B) = \neg A \cap \neg B$

LHS: $A \cup B = \{C, D, E, F, G, H\}$

$\neg(A \cup B) = \{A, B, I, J\}$

RHS: $\neg A = \{A, B, G, H, I, J\}$

$\neg B = \{A, B, C, D, I, J\}$

$\neg A \cap \neg B = \{A, B, I, J\}$

LHS = RHS

3 points: Student correctly creates LHS and RHS step by step and the result of each side equals {A, B, I, J}. Students can use any of the three different notations.

-1 point: Student doesn't create LHS and RHS step by step (-0.5 points if only one of them is not created step by step)

-2 points: Student doesn't give correct results of LHS and RHS (-1 point if only one of them is wrong)

0 point: If no answer or irrelevant answer.

Q7 Provide three examples of 'special purpose' databases - what would they store, what might their interface be? No need for diagrams - but do mention why they would be 'special'. Hint - we went through a molecule DB in class.

Examples could be in these domains: genealogy, stocks, bitcoin, videogame, fingerprint, astronomy...

1pt per example: The student gets the full score if he/she explain **what** it stores, what the **interface** is (eg. might contain a dedicated viewer or a search engine or calculator, etc), and **why** it is special (ie. why it is more than just a standard relational DB) for that example. If any part is missing in an example, the student only gets **0.5** for that example.

Q8 An(y) entity can be described by a set of characteristics (ie. "columns") - that is what 'data' actually is, as you know.

In principle, practically ANY entity might have *in common*, some characteristics/properties/columns with ANY OTHER entity!

- Why is a raven like a writing desk? Possible answers: both are **black, have feet, have flat surfaces, are shiny**, etc.
- What's common between a flash drive and an auditorium? A: both **contain something** (data/information/content, vs. people).
- What's common between a baseball game and a (theatrical) play? A: both **happen in real-time ('live') in front of an audience**.

Q9 'Joins' occupy a special place in relational DBs. Discuss them: what they are, how the table notion facilitates them.

(1 point) Clearly defines joins as operations that combine rows from two or more tables based on a related column (typically using a foreign key).

(1 point) Mentions types of joins (e.g., inner, left, right) or gives a concrete example.

(1 point) Discusses how the consistent structure of tables (e.g., shared columns, referential integrity) allows efficient and meaningful joining.

Q10

[2 points]

How does the SQL 'EXISTS' operator work?

- **2 points:** Clearly explains that EXISTS checks for the existence of rows returned by a subquery. **Explicitly notes that it returns TRUE if the subquery returns at least one row, otherwise, it returns FALSE.**
- **1 point:** Mentions it checks for the existence of rows, but misses a key detail (e.g., doesn't specify the TRUE/FALSE behavior or talks vaguely).
- **0 points:** Incorrect or no answer provided.

[1 point]

What makes a 'correlated subquery' become correlated?

- **1 point:** Clearly states that a correlated subquery references one or more columns from the outer ('main') query. **Need to** additionally mention that the subquery is evaluated once per row of the outer query.
- **0.5 points:** Demonstrates partial understanding but is vague or incomplete (e.g., mentions a dependency or relationship between subquery and outer query without clearly stating column reference or per-row evaluation).
- **0 point:** Incorrect or no answer provided.

Q11

[1 point] Identify FROM + JOIN construct

- **1 point:** States that FROM/JOIN correspond to **iteration** across collections, typically implemented as nested loops (outer loop over rows of the first table, inner loop over rows of the second, or an equivalent join algorithm). In other words, FROM+JOIN is equivalent to a **for()** loop.
- **0.5 point:** Identifies "loop/iteration" but does not acknowledge nested loops / pairing logic.

[1 point] Identify WHERE construct

- **1 point:** States that WHERE is a **conditional** filter, i.e., an if (or similar) that keeps rows only when the predicate is true. IOW, WHERE is equivalent to an **if()** statement.
- **0.5 point:** Mentions "filter" or "condition" but omits the control-flow term (if, case, etc.).

[1 point] Provide coherent pseudocode combining the two

- **1 point:** Supplies pseudocode that: 1) initializes a result container; 2) iterates (via one or more loops) over the table(s); 3) applies an if condition to implement the WHERE; 4) appends or outputs matching rows/selected fields. The order must be logically correct (loop(s) → if → collect).

Acceptable too [doesn't have to be verbatim, but shows the nesting]:

for each row

if a condition (can be a single one or an expression connected by Booleans) is true

output/print the row's requested columns

- **0.5 point:** Minor syntax/ordering issue or omits the result-collection step, but the logic is still clear.

Q12

[1 point] Why is 'closure' a BFD for table operations?

- **1 point:** Student correctly explains that closure ensures results stay within the same domain, essential for predictability and chaining operations (table operations such as SELECT, UNION etc result in a table as output, i.e. the **TYPE is PRESERVED**). Acceptable phrasing includes: "So you can combine rows without breaking the system," or "You stay inside the table."
- **0.5 points:** Students give a vague or partially correct answer (e.g., "Because it works" or "It makes things easier") without explaining why closure matters.
- **0 points:** No answer or irrelevant answer.

[1 point] What is an operation that BREAKS closure on square matrices?

- **1 point:** Student gives a valid operation that takes square matrices and produces something outside that set. Acceptable examples: Taking the **determinant** (returns a scalar, not a matrix), **Eigenvalue decomposition** (returns vectors or non-square structures), or the **rank** operation.
- **0.5 points:** Student gives an operation on matrices but doesn't explain why it breaks closure or is ambiguous (e.g., "inverse", which preserves closure on invertible matrices).
- **0 points:** Incorrect or missing.

[1 point] What is an operation that PRESERVES closure on colors ([r,g,b])?

- **1 point:** Student gives a valid operation that maps one or more colors to another [r,g,b] color. Acceptable examples: **Addition** with clipping (e.g., [r1,g1,b1] + [r2,g2,b2], clipped to max 255), Averaging colors (blur), Scalar multiplication with clipping (brightening/darkening). Also - contrast enhancement, sharpening, median filter...
- **0.5 points:** Partially valid operation (e.g., "adding colors" without noting clipping or staying in bounds)
- **0 points:** Irrelevant or invalid operation (e.g., converting to grayscale without returning [r,g,b]).